macOS Dev Setup (No Docker)

A lighter local setup for day-to-day development on macOS — same ten modules, no containers, using a Python virtual environment instead. Use this for fast iteration; use the Docker Compose runbook (other tab) for the full-stack demo and anything you'll show judges.

1. Prerequisites (macOS)

1. **Homebrew** — if not already installed: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
2. **Python 3.11** — `brew install python@3.11` (match the version the Docker image uses, so behavior stays consistent between dev and the demo stack).
3. **PostgreSQL 16 + pgvector** — `brew install postgresql@16 pgvector` (pgvector is a Postgres extension; Homebrew's formula builds it against your local Postgres).
4. **Git** — usually preinstalled on macOS; confirm with `git --version`.
5. An **Anthropic API key** (see the other tab's note on alternatives if you don't have one yet).

No Docker Desktop is needed for this workflow — it stays reserved for the full-stack demo version.

2. Clone the repo and create an isolated venv

`venv` keeps this project's dependencies separate from anything else on your Mac — no version conflicts with other Python projects, and an easy way to wipe and start clean if something breaks.

```
git clone <repo-url> && cd fcrm-workbench

# Create the virtual environment (once)
python3.11 -m venv .venv

# Activate it (every new terminal session)
source .venv/bin/activate

# Your prompt now shows (.venv) — confirm you're using the venv's Python
which python
# → .../fcrm-workbench/.venv/bin/python
```

To leave the venv later: `deactivate`. `.venv/` should already be in `.gitignore` — never commit it.

3. Install dependencies into the venv

With the venv activated (`(.venv)` showing in your prompt):

```
pip install --upgrade pip
pip install -r requirements.txt
```

This is the same `requirements.txt` the Docker image uses (LangGraph, langchain-anthropic, FastAPI, SQLAlchemy, psycopg2-binary, pgvector, Ragas/DeepEval, Langfuse, etc.) — one dependency list shared by both setups keeps dev and demo behavior identical. If a package fails to build (a native extension like `psycopg2-binary` occasionally does on Apple Silicon), `brew install postgresql@16` first (step 1) usually resolves it, since it provides the Postgres headers the build needs.

4. Set up local Postgres with pgvector

```
brew services start postgresql@16

# Create the database and a user matching the Docker version's defaults
createdb fcrm_workbench
psql fcrm_workbench -c "CREATE USER fcrm WITH PASSWORD 'localdev';"
psql fcrm_workbench -c "GRANT ALL PRIVILEGES ON DATABASE fcrm_workbench TO
fcrm;"

# Enable the pgvector extension (needed for M3's policy retrieval)
psql fcrm_workbench -c "CREATE EXTENSION IF NOT EXISTS vector;"

# Run the same schema the Docker setup runs automatically on first boot
psql fcrm_workbench -f infra/init-db/001_schema.sql
```

To stop Postgres later: `brew services stop postgresql@16`. To reset the database during development: `dropdb fcrm_workbench` and repeat the block above.

Troubleshooting — `extension "vector" is not available`: Homebrew's `pgvector` formula builds against whichever `pg_config` is first on your `PATH` at install time — if that isn't `postgresql@16`'s, the extension files never land where Postgres 16 looks for them. Fix:

```
# Point at postgresql@16 explicitly, then reinstall pgvector against it
export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"
brew reinstall pgvector

# Confirm pg_config now resolves to postgresql@16
pg_config --version

# Retry
psql fcrm_workbench -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Still failing? Build pgvector from source against the exact `pg_config` binary:

```
git clone --branch v0.7.4 https://github.com/pgvector/pgvector.git
cd pgvector
export PG_CONFIG=/opt/homebrew/opt/postgresql@16/bin/pg_config
make && make install
cd .. && rm -rf pgvector
```

(Intel Macs: swap `/opt/homebrew` for `/usr/local` throughout.)

5. Local `.env` configuration

The only difference from the Docker version's `.env` is the database host — `localhost` instead of the Compose service name `db`:

```
# .env (local dev)
ANTHROPIC_API_KEY=sk-ant-...
DATABASE_URL=postgresql://fcrm:localdev@localhost:5432/fcrm_workbench

# Leave these blank for local dev — see section 8 below
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
```

Keep this as a separate `.env.local` (git-ignored, same as `.env`) if you want to switch between local dev and the Docker stack without editing the same file back and forth.

6. Run the app locally

With the venv active and Postgres running:

```
uvicorn app.orchestrator.main:app --reload --port 8000
```

`--reload` restarts the server automatically as you edit code — the equivalent of the Docker version's `volumes` live-mount, but faster since there's no container filesystem layer in the way. Visit `http://localhost:8000/docs` for the same auto-generated API docs as the Docker version.

Run each module's own tests directly too, without spinning up the whole app:

```
pytest tests/ -k extraction   # e.g. just M2's tests while you're working on
it
```

7. Eval/observability tracing without a local Langfuse container

Running the Langfuse server itself needs Docker, so for local dev pick one of:

- **Skip tracing locally** — leave `LANGFUSE_PUBLIC_KEY` / `LANGFUSE_SECRET_KEY` blank in `.env`; guard the Langfuse calls in code (`if settings.langfuse_public_key: ...`) so the app runs fine without it. Good enough for day-to-day module development.
- **Point at Langfuse Cloud instead** — sign up for a free Langfuse Cloud project, drop its public/secret keys into `.env`, and traces show up in their hosted dashboard with zero local infrastructure.
- **Run the golden-case regression suite directly** with `pytest tests/eval_regression.py` — Ragas/DeepEval scoring doesn't need Langfuse at all, only tracing/dashboards do.

Whichever you pick for dev, run the real Docker Compose stack (with the local Langfuse container) at least once before the demo, so the trace dashboard you show judges is the one from section 8 of the Docker runbook, not Langfuse Cloud.

8. When to switch back to the Docker stack

This venv setup is for fast, individual iteration on one module at a time. Switch to the Docker Compose runbook (other tab) before:

- Testing M0's orchestration across all ten modules together — closer to how it'll actually run.
- Anything touching Langfuse tracing you'll show in the submission.
- Final integration testing, demo rehearsal, and the actual submission — judges should see the containerized, infrastructure-as-code version, since that's what's scored under deployment/production readiness.

Both setups read the same `requirements.txt`, the same `infra/init-db/001_schema.sql`, and the same module code under `app/` — nothing here forks the codebase, only how it's run locally.